

Legacy Memory APIs

**ConversationBufferMemory,
ConversationBufferWindowMemory & ConversationChain**

Older LangChain applications commonly used:

ConversationBufferMemory

ConversationBufferWindowMemory

ConversationChain

Important Note

These are **legacy APIs**.

They are still useful because:

- Students may see them in older tutorials.
- Students may see them in existing projects.
- They help us understand how LangChain memory evolved.

Current LangChain documentation treats these classic memory abstractions as deprecated.

ConversationChain is also deprecated in favour of newer approaches such as:

RunnableWithMessageHistory

1. Quick Revision — What Did We Do Manually?

In previous sessions, we manually maintained conversation history.

User Input



Add HumanMessage



History



Send History to Model



Get AIMessage



Add AIMessage

We ourselves handled three important operations:

STORE



UPDATE



PASS

We stored the history.

We updated the history.

We passed the history to the model.

Now imagine if LangChain could perform some of these operations for us.

That was the idea behind the older:

Memory APIs

2. ConversationBufferMemory

ConversationBufferMemory stores conversation history in a buffer.

In simple words:

It keeps the entire conversation history without additional processing.

3. Installation

For the legacy memory APIs:

```
pip install langchain_classic
```

Import:

```
from langchain_classic.memory import  
ConversationBufferMemory
```

4. Create Buffer Memory

```
memory = ConversationBufferMemory()
```

Conceptually:

```
memory
```



Empty Conversation Buffer

Initially, no conversation is stored.

5. Save Conversation into Legacy Memory

Legacy memory provides:

```
save_context()
```

Example:

```
memory.save_context(  
    {  
        "input": "My name is Durga."  
    },  
    {
```

```
        "output": "Nice to meet you, Durga."  
    }  
)
```

Concept:

Human Input

+

AI Output

↓

save_context()

↓

Memory

6. What Does save_context() Do?

It stores one interaction.

Example:

Human:

My name is Durga.

AI:

Nice to meet you, Durga.

This complete interaction is added to memory.

7. Read Stored Memory

Legacy memory provides:

```
load_memory_variables()
```

Example:

```
result = memory.load_memory_variables({})
```

Then:

```
print(result)
```

A legacy memory object normally exposes the conversation using a memory key such as:

```
history
```

8. Basic Pattern

```
memory = ConversationBufferMemory()
```

```
memory.save_context(  
    {"input": "User Message"},  
    {"output": "AI Response"}  
)
```

```
result = memory.load_memory_variables({})
```

```
print(result)
```

Easy memory:

SAVE



```
save_context()
```

READ



```
load_memory_variables()
```

9. Program 1 — ConversationBufferMemory: Save and Load History

```
from langchain_classic.memory import  
ConversationBufferMemory
```

```
# =====
```

```
# CREATE LEGACY MEMORY
```

```
# =====
```

```
memory = ConversationBufferMemory()
```

```
# =====
```

```
# SAVE FIRST INTERACTION
```

```
# =====
```

```
memory.save_context(  
    {  
        "input": "My name is Durga."  
    },  
    {  
        "output": "Nice to meet you, Durga."  
    }  
)
```

```
# =====
```

```
# SAVE SECOND INTERACTION
```

```
# =====
```

```
memory.save_context(  
    {  
        "input": "I teach Python."  
    },  
    {  
        "output": "That's great."  
    }  
)
```

```
# =====
```

```
# READ MEMORY
```

```
# =====
```

```
result = memory.load_memory_variables({})
```

```
print("STORED MEMORY")
```

```
print("-----")
```

```
print(result)
```

10. Output

```
{
```

```
  'history':
```

```
    "Human: My name is Durga.
```

```
    AI: Nice to meet you, Durga.
```

```
    Human: I teach Python.
```

```
    AI: That's great."
```

```
}
```

Conceptually:

Interaction 1



Stored

Interaction 2



Stored



Complete Buffer

11. ConversationBufferMemory Stores Full History

Suppose we have:

Conversation 1

Conversation 2

Conversation 3

Conversation 4

Conversation 5

ConversationBufferMemory keeps:

Conversation 1

Conversation 2

Conversation 3

Conversation 4

Conversation 5

It does not automatically restrict the history to only recent conversations.

12. Default History Format

By default, the legacy memory may expose history in formatted form.

For example:

Human: My name is Durga.

AI: Nice to meet you.

Human: I teach Python.

AI: Great.

But sometimes we may want actual message objects.

For that, we can use:

```
return_messages=True
```

13. ConversationBufferMemory with return_messages=True

```
memory = ConversationBufferMemory(  
    return_messages=True  
)
```

Now the history can be returned as message objects.

Conceptually:

HumanMessage

AIMessage

HumanMessage

AIMessage

instead of only one formatted string.

14. Example Result

The result may look like:

```
{  
  "history": [  
    HumanMessage(  
      content="My name is Durga."  
    ),  
  
    AIMessage(  
      content="Nice to meet you."  
    ),  
  
    HumanMessage(  
      content="I teach Python."  
    ),  
  ]  
}
```

```
    AIMessage(  
        content="Great."  
    )  
]  
}
```

This connects nicely with the concepts we already learned:

HumanMessage

+

AIMessage

15. Printing Message Objects Clearly

for message in result["history"]:

```
    print(  
        type(message).__name__,  
        ":",  
        message.content  
    )
```

Output may look like:

HumanMessage : My name is Durga.

AIMessage : Nice to meet you.

HumanMessage : I teach Python.

AIMessage : Great.

16. return_messages=True

Easy difference:

Without return_messages=True

Formatted History

Example:

Human: ...

AI: ...

With return_messages=True

HumanMessage(...)

AIMessage(...)

17. clear()

Suppose we want to remove the complete stored conversation.

Legacy memory supports:

memory.clear()

18. Why Might We Clear Memory?

Example:

For some time, we are discussing:

Cricket

Later, we completely change the topic to:

Health Insurance

If the old cricket discussion is no longer useful, we may decide to clear the old memory.

Concept:

Old Conversation



`memory.clear()`



Empty Memory

19. Program — Clear Complete Memory

```
from langchain_classic.memory import  
ConversationBufferMemory
```

```
memory = ConversationBufferMemory()
```

```
memory.save_context(  
    {  
        "input": "My name is Durga."  
    },  
    {  
        "output": "Nice to meet you."  
    }  
)
```

```
memory.save_context(  
    {  
        "input": "I teach Python."  
    },  
    {  
        "output": "Great."  
    }  
)
```

```
)
```

```
print("BEFORE CLEAR")
```

```
print("-----")
```

```
print(
```

```
    memory.load_memory_variables({})
```

```
)
```

```
# Clear complete memory
```

```
memory.clear()
```

```
print("\nAFTER CLEAR")
```

```
print("-----")
```

```
print(
```

```
memory.load_memory_variables({})  
)
```

20. Output Before clear()

BEFORE CLEAR

```
{  
  'history':  
    'Human: My name is Durga.  
    AI: Nice to meet you.  
    Human: I teach Python.  
    AI: Great.'  
}
```

21. Output After clear()

AFTER CLEAR

```
{
```

```
'history': "  
}
```

So:

```
memory.clear()
```



Removes Stored Conversation

22. Main Problem with Full Buffer Memory

ConversationBufferMemory keeps the complete conversation.

If the conversation becomes very large, this can create problems.

Problem 1 — More Context Usage

More History



More Model Context Used

Problem 2 — More Tokens

More History



More Input Tokens



Potentially More Cost

Problem 3 — Latency

Large History



Larger Model Input



Potentially More Latency

To deal with this type of problem, older LangChain provided:

ConversationBufferWindowMemory

23. ConversationBufferWindowMemory

ConversationBufferWindowMemory keeps only a recent conversational window instead of exposing the entire conversation buffer.

This is similar to the manual concept we already learned:

```
recent_history = history[-MAX_MESSAGES:]
```

Manual approach:

Full History



Take Recent Part



Send Recent History

Legacy Window Memory:

Conversation



ConversationBufferWindowMemory



Recent Window

24. Import ConversationBufferWindowMemory

```
from langchain_classic.memory import (  
    ConversationBufferWindowMemory  
)
```

25. Create Window Memory

Example:

```
memory = ConversationBufferWindowMemory(  
    k=2  
)
```

Here:

k



Controls Recent Conversation Window

In these notes:

k = Number of Recent Conversation Turns

So:

$k=2$

means keeping the recent two conversational turns in the active window.

26. Messages vs Turns

Remember:

One normal conversation turn may contain:

Human Message

+

AI Message

Therefore:

1 Turn



Human + AI

So when:

k=2

we are talking about recent conversation turns, not simply two individual message objects.

27. Program — ConversationBufferWindowMemory

```
from langchain_classic.memory import (
    ConversationBufferWindowMemory
)

# =====

# CREATE WINDOW MEMORY

# =====

memory = ConversationBufferWindowMemory(
    k=2
)

# =====
```

```
# FIRST INTERACTION
```

```
# =====
```

```
memory.save_context(  
    {  
        "input": "My name is Durga."  
    },  
    {  
        "output": "Nice to meet you."  
    }  
)
```

```
# =====
```

```
# SECOND INTERACTION
```

```
# =====
```

```
memory.save_context(  
    {  
        "input": "I teach Python."
```

```
    },  
    {  
        "output": "Great."  
    }  
)  
  

```

```
# =====  
# THIRD INTERACTION  
# =====  
  

```

```
memory.save_context(  
    {  
        "input": "I live in Hyderabad."  
    },  
    {  
        "output": "Okay."  
    }  
)  
  

```

```
# =====  
# READ ACTIVE WINDOW  
# =====  
  
result = memory.load_memory_variables(  
    {}  
)  
  
print("WINDOW MEMORY")  
print("-----")  
print(result)
```

28. Understanding k=2

We saved three interactions:

Interaction 1

My name is Durga.



Nice to meet you.

Interaction 2

I teach Python.



Great.

Interaction 3

I live in Hyderabad.



Okay.

But:

$k=2$

means the active memory window focuses on the recent two turns.

Conceptually:

Older Turn



Outside Active Window

Recent Turn 1



Inside Window

Recent Turn 2



Inside Window

29. Buffer Memory vs Window Memory

Concept	ConversationBufferMemory	ConversationBufferWindow Memory
Main idea	Keep full conversation	Keep recent conversation window
Old conversations	Remain in exposed buffer	Fall outside recent active window
History growth	Can keep growing	Active history is restricted
Main parameter	No k required	Uses k
Use case	Small/full history	Recent-history approach

30. Simple Memory Formula

ConversationBufferMemory

Conversation 1

Conversation 2

Conversation 3

Conversation 4

Conversation 5



Keep Everything

ConversationBufferWindowMemory

Conversation 1

Conversation 2

Conversation 3

Conversation 4

Conversation 5



Keep Recent Window

31. return_messages=True with Window Memory

return_messages=True can also be used with ConversationBufferWindowMemory.

Example:

```
memory = ConversationBufferWindowMemory(  
    k=2,  
    return_messages=True  
)
```

Then the recent conversation history can be exposed as message objects.

Conceptually:

HumanMessage

AIMessage

HumanMessage

AIMessage

instead of only formatted history text.

32. Connection with Our Previous Manual Approach

Previously, we manually maintained:

```
history = []
```

Then:

```
recent_history = history[-6:]
```

Concept:

Full History



Python List Slicing



Recent History

ConversationBufferWindowMemory provided an older LangChain abstraction for a similar recent-history idea.

33. Manual History vs Legacy Memory

Manual History

We ourselves perform:

STORE

UPDATE

READ

PASS

using things such as:

`messages.append(...)`

Legacy Memory

The memory object provides operations such as:

```
memory.save_context(...)
```

and:

```
memory.load_memory_variables(...)
```

So the memory abstraction manages conversation history for us.

34. Main Legacy Memory Methods

Save Conversation

```
memory.save_context(  
    {"input": "..."},  
    {"output": "..."}  
)
```

Read Conversation

```
memory.load_memory_variables({})
```

Clear Conversation

```
memory.clear()
```

35. Easy Method Table

Method	Purpose
save_context()	Store one interaction

`load_memory_variables()` Read stored memory

`clear()` Remove stored memory

36. Why Learn These Legacy APIs?

Even though they are legacy APIs, they help us understand the evolution of LangChain memory.

Learning them helps when:

Reading Old Tutorials



Understanding Existing Code



Understanding Why New APIs Exist

Later, when we learn newer APIs such as:

`RunnableWithMessageHistory`

we can better understand what problem they are solving.

37. Important Key Points

1. `ConversationBufferMemory` is a legacy LangChain memory API.
2. It stores the complete conversation history in a buffer.
3. `save_context()` stores a Human/AI interaction.

4. `load_memory_variables()` reads the stored memory.
5. Legacy memory commonly exposes conversation data using a key such as `history`.
6. By default, history may be presented in formatted form.
7. `return_messages=True` exposes the history as message objects.
8. `clear()` removes stored conversation history.
9. Full buffer memory can increase context usage, token usage, cost and latency as history grows.
10. `ConversationBufferWindowMemory` keeps a recent conversation window.
11. Its `k` value controls the recent conversation window in terms of turns in these examples.
12. `return_messages=True` can also be used with window memory.
13. These APIs are useful mainly for understanding older LangChain applications and tutorials.

38. Common Mistake — Buffer Means Permanent Storage

Do not think:

`ConversationBufferMemory`

=

Permanent Database

The word:

Memory

does not automatically mean permanent persistent storage.

39. Common Mistake — $k=2$ Means Two Tokens

Wrong.

$k=2$

does not mean:

2 Tokens

In these notes, it represents:

2 Recent Conversation Turns

40. Common Mistake — Window Memory Means Full History

Wrong.

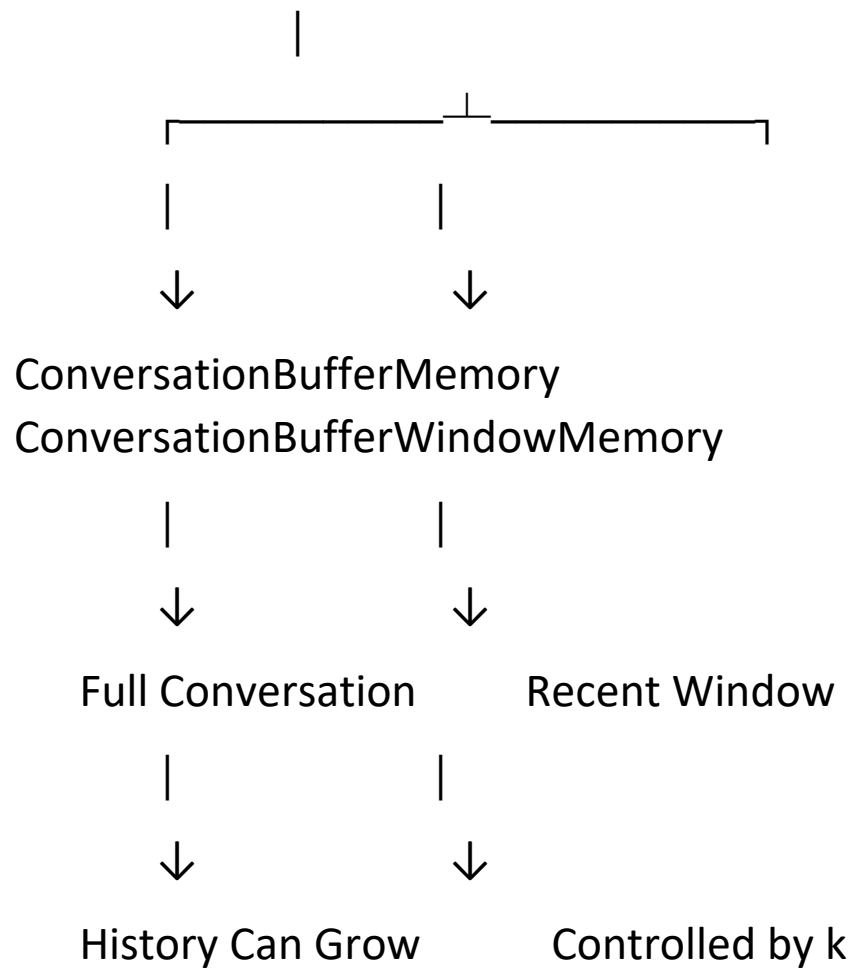
The purpose of:

ConversationBufferWindowMemory

is to work with a limited recent conversation window.

41. Master Architecture

LEGACY MEMORY APIs



42. Evolution of Our Learning

Step 1 — Manual History

messages = []



Append Human



Invoke Model



Append AI

Step 2 — Manual Recent Window

history



history[-6:]



Recent History

Step 3 — Legacy Memory APIs

ConversationBufferMemory

ConversationBufferWindowMemory

Step 4 — Modern History APIs

Later we move towards approaches such as:

RunnableWithMessageHistory

43. Final Memory Formula

Full Buffer

ConversationBufferMemory



FULL HISTORY

Window Buffer

ConversationBufferWindowMemory



RECENT HISTORY WINDOW

44. One-Line Conclusion

ConversationBufferMemory stores the full conversation buffer, while ConversationBufferWindowMemory keeps a recent conversational window; both belong to the older LangChain memory style that students may still see in legacy tutorials and applications.